Tutorato AsD ~ L10 — 21/05/2026

Esercizio 1. Il *trasposto* di un grafo orientato $G = (V, E)$ è il grafo $G^T = (V, E^T)$, dove $E^T = \{(v, u) \in V \times V : (u, v) \in E\}$. Quindi, G^T è G con tutti i suoi archi invertiti. Descrivete un algoritmo con tempo di esecuzione $O(n + m)$ per calcolare G^T a partire da G, assumendo che i grafi siano rappresentati tramite liste di adiacenza.

$G.Adj[u] \longrightarrow G^T.Adj[u]$





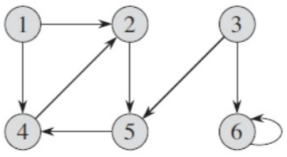
$\forall u \in G$ $G^T.Adj[u]$ inizialmente vuota $O(n)$

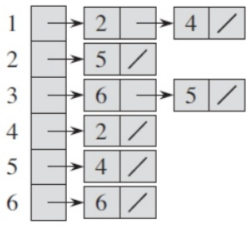
```
for each u ∈ G.V
    for each v ∈ G.Adj[u]
        add u to G^T.adj[v]
```

$O(m)$

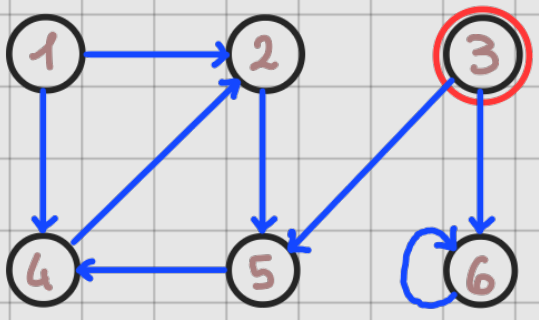
totale: $O(n+m)$

Esercizio 2. Calcolate i valori d e π che si ottengono effettuando una visita in ampiezza del seguente grafo utilizzando il vertice 3 come sorgente.

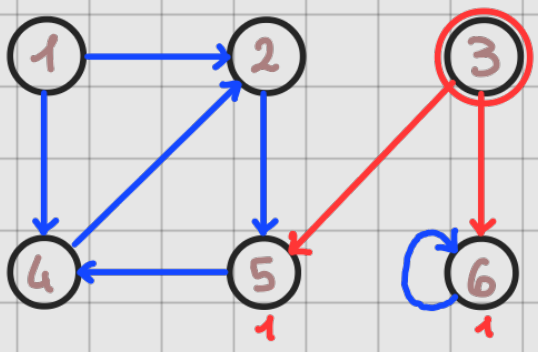




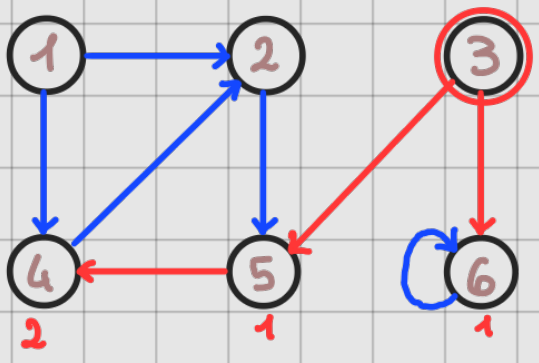
I.



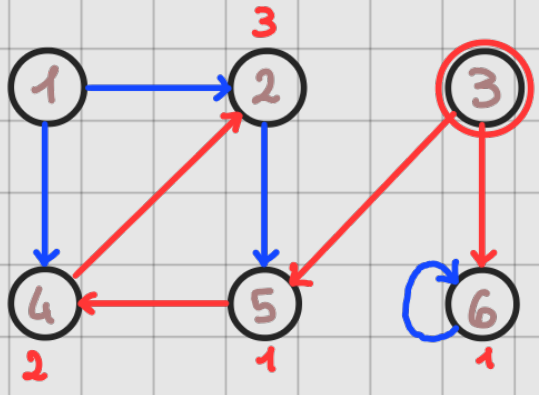
II.



III.



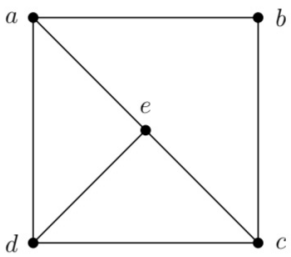
IV.



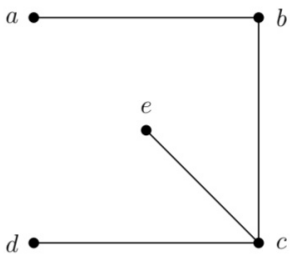
S:

v	d	π
1	∞	NIL
2	3	4
3	0	NIL
4	2	5
5	1	3
6	1	3

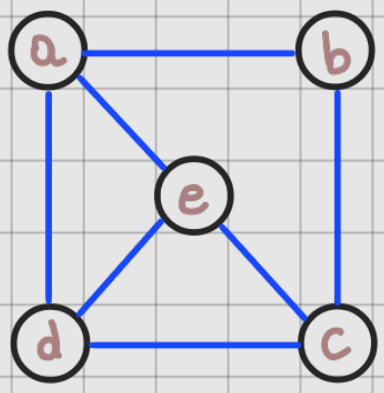
Esercizio 3. Dati il grafo G e l'albero T in figura, spiegate se T può essere ottenuto come albero BFS da una visita in ampiezza di G.

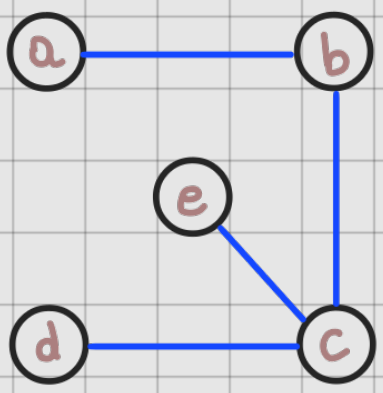


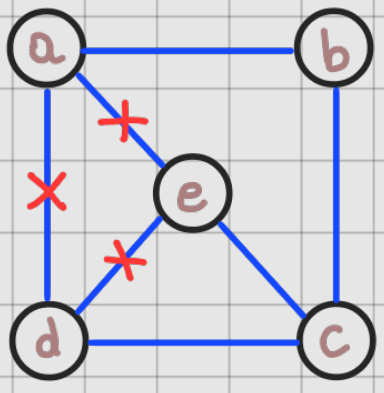
G



T







Candidati source: b, c

S

a) → * → * → *

b) → c → a

c) → b → d → e

d) → * → * → *

e) → * → * → *

Esercizio 4. Sia $G = (V, E)$ un grafo non-diretto e connesso. L'*eccentricità* $\varepsilon(u)$ di un vertice $u \in V$ è la massima distanza tra u ed un qualsiasi altro vertice di G. Dato $s \in V$, fornite un algoritmo con tempo di esecuzione lineare (i.e., $O(|V| + |E|)$) che calcoli $\varepsilon(s)$.

$$\varepsilon(u) = \max_{v \in V} \delta(u, v)$$

Lanciamo BFS con sorgente s.

BFS calcola $v.d = \delta(s, v) \quad \forall v \in V$ $O(n+m)$

$$\varepsilon(s) = \max_{v \in V} v.d$$ $O(n)$

totale: $O(n+m)$

Esercizio 5. Ci sono due tipi di lottatori professionisti: “buoni” e “cattivi”. Fra una coppia qualsiasi di lottatori professionisti ci può essere o no rivalità. Supponete di avere n lottatori e una lista di r coppie di lottatori fra i quali c’è rivalità. Create un algoritmo con tempo di esecuzione $O(n+r)$ che determini se sia possibile designare alcuni lottatori come buoni e tutti gli altri come cattivi, in modo che la rivalità sia sempre fra un buono e un cattivo. Se tale designazione è possibile, il vostro algoritmo dovrebbe effettuarla.

$G = (V, E)$ $\quad V \rightarrow$ lottatori

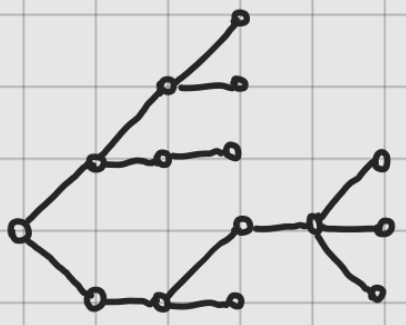
$(u,v) \in E \iff u$ & v sono rivali $\quad |E| = r$

Voglio quindi un grafo con numero cromatico

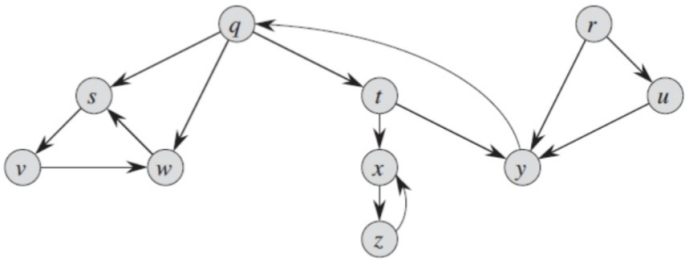
$\chi(G) \leq$ numero di insiemi $= 2$.

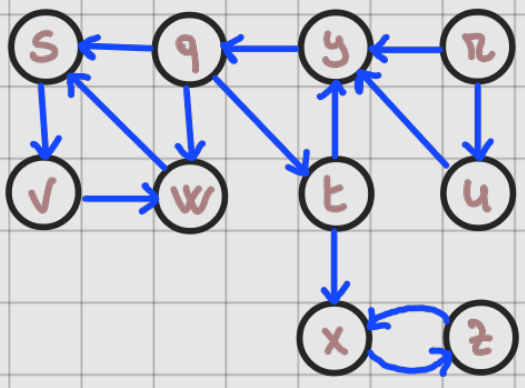
Voglio cioé testare se é bipartito.

Lo posso fare con BFS in $O(n+r)$



Esercizio 6. Illustrate come funziona una visita in profondità del seguente grafo, supponendo che i vertici siano inizialmente ordinati alfabeticamente e che ogni lista di adiacenza sia ordinata alfabeticamente.

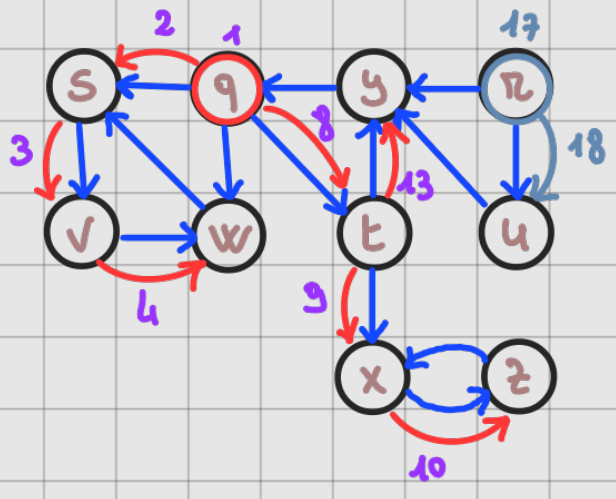


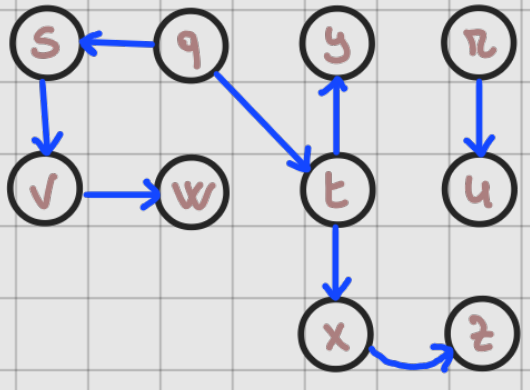


Adj

q	s, t, w
r	u, y
s	v
t	x, y
u	y
v	w
w	s
x	z
y	q
z	x

	v.d	v.f	v.π
q	1	16	NIL
r	17	20	NIL
s	2	7	q
t	8	15	q
u	18	19	r
v	3	6	s
w	4	5	v
x	9	12	t
y	13	14	t
z	10	11	x





Esercizio 7. Riscrivete la procedura DFS utilizzando una pila (stack) per eliminare la ricorsione.

```
DFS(G)
1  for each vertex u ∈ G.V
2      u.color = WHITE
3      u.π = NIL
4  time = 0
5  for each vertex u ∈ G.V
6      if u.color == WHITE
7          DFS-VISIT(G, u)

DFS-VISIT(G, u)
1  time = time + 1                  // white vertex u has just been discovered
2  u.d = time
3  u.color = GRAY
4  for each vertex v in G.Adj[u]    // explore each edge (u, v)
5      if v.color == WHITE
6          v.π = u
7          DFS-VISIT(G, v)
8  time = time + 1
9  u.f = time
10 u.color = BLACK                  // blacken u; it is finished
```

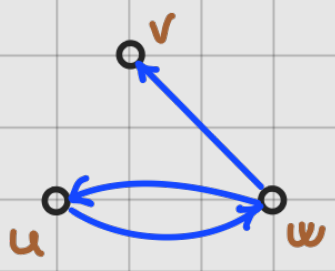
```
DFS-VISIT-STACK(G,u):
    S = ∅
    PUSH(S,u)
    time = time + 1
    u.d = time
    u.color = gray
    while S ≠ ∅
        u = S.top
        v = FIRST-WHITE(G,u)
        if v = NIL :        Lista di u completamente esplorata
            POP(S)
            u.color = black
            time = time + 1
            u.f = time
        else :              v è il prossimo vicino bianco di u
            V.π = u
            time = time + 1
            V.d = time
            V.color = gray
            PUSH(S,V)
```

```
FIRST-WHITE(G,u)
    for each v ∈ G.Adj[u]
        if v.color = white
            return v
    return NIL
```

Esercizio 8. Trovate un controesempio alla congettura che, se esiste un cammino da u a v in un grafo diretto G e se $u.d < v.d$ in una visita in profondità di G, allora v è un discendente di u nella foresta DFS risultante.

$u \rightsquigarrow v$ & $u.d < v.d$ $\Rightarrow$ v discendente di u in albero DFS

G



Ordine su V: w u v

Ordine lista di w: u v

	d	f	π
u	2	3	w
v	4	5	w
w	1	6	NIL

- $u.d < v.d$ ✓
  2 < 4
- $u \rightsquigarrow v$ ✓
  u → w → v

✗ u & v fratelli

Esercizio 9. Modificate lo pseudocodice per la visita in profondità di un grafo diretto in modo che assegni il tipo corretto ad ogni arco.

Sugg.: Cosa succede se, alla prima ispezione di (u, v), v è nero e $u.d < v.d$?

Classification of edges

You can obtain important information about a graph by classifying its edges during a depth-first search. For example, Section 20.4 will show that a directed graph is acyclic if and only if a depth-first search yields no "back" edges (Lemma 20.11).

The depth-first forest G_π produced by a depth-first search on graph G can contain four types of edges:

1. ***Tree edges*** are edges in the depth-first forest G_π. Edge (u, v) is a tree edge if v was first discovered by exploring edge (u, v).
2. ***Back edges*** are those edges (u, v) connecting a vertex u to an ancestor v in a depth-first tree. We consider self-loops, which may occur in directed graphs, to be back edges.
3. ***Forward edges*** are those nontree edges (u, v) connecting a vertex u to a proper descendant v in a depth-first tree.
4. ***Cross edges*** are all other edges. They can go between vertices in the same depth-first tree, as long as one vertex is not an ancestor of the other, or they can go between vertices in different depth-first trees.

ARCO CHE HA SCOPERTO V PER LA PRIMA VOLTA

ARCO CHE TORNA VERSO UN ANTENATO ANCORA IN ESPLORAZIONE

ARCO CHE SALTA VERSO UN DISCENDENTE GIÁ COMPLETATO

ARCO VERSO UN VERTICE GIÁ COMPLETATO, IN UN ALTRO RAMO

Classifico (u,v) in base al colore di v durante la prima ispezione di (u,v):

1. v BIANCO → (u,v) arco d'albero
2. v GRIGIO → (u,v) arco all'indietro
3. v NERO → (u,v) arco in avanti o trasversale
   - ↳ Se v NERO & $u.d < v.d$ → (u,v) arco in avanti

Corollary 20.8 (Nesting of descendants' intervals)
Vertex v is a proper descendant of vertex u in the depth-first forest for a (directed or undirected) graph G if and only if $u.d < v.d < v.f < u.f$.

- ↳ Altrimenti arco trasversale

```
DFS-VISIT(G,u)
    time = time + 1
    u.d = time
    u.color = gray
    for each v ∈ G.Adj[u]
        if v.color = white:
            print "arco d'albero"
            v.π = u
        else if v.color = gray:
            print "arco all'indietro"
        else if v.d > u.d :
            print "arco in avanti"
        else:
            print "arco trasversale"
    u.color = black
    time = time + 1
    u.f = time
```

Esercizio 10. Descrivete un algoritmo che determini in tempo $O(n)$ se un grafo *non-diretto* G con n vertici contiene un ciclo oppure no.

Usiamo DFS fermandoci se incontriamo un arco all'indietro

1. G è aciclico

   Una foresta con n vertici e p componenti connesse

   ha esattamente n - p archi.

   Quindi: $m \leq n - 1$ (con $p \geq 1$)

   ↳ DFS richiede $O(n+m) = O(n)$

2. G contiene un ciclo. Ci fermiamo prematuramente

   ispezionando l'arco all'indietro (u, v)

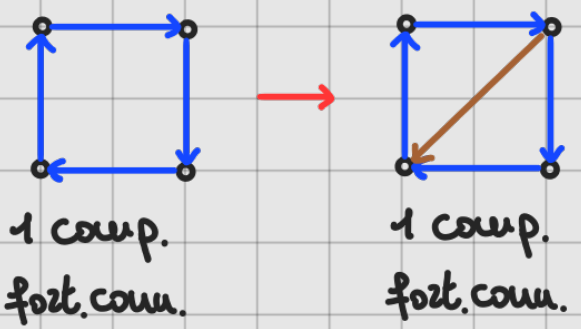
   ↳ archi considerati fino a quel momento formano una foresta.

   ↳ Come sopra $O(n+m) = O(n)$

Esercizio 11. Come cambia il numero delle componenti fortemente connesse di un grafo se viene aggiunto un nuovo arco?

comp. fortemente connesse dopo aggiunta di un arco (u,v) può:

1. rimanere invariato:

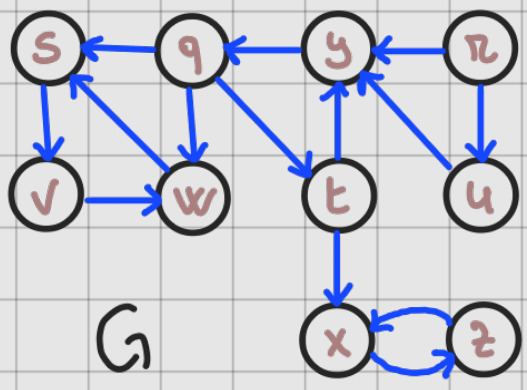


2. decrescere:



Non può crescere: aggiungendo un arco, vertici che erano vicendevolmente raggiungibili rimangono vicendevolmente raggiungibili.

Esercizio 12. Calcolate le componenti fortemente connesse del grafo dell'esercizio 6 utilizzando l'algoritmo STRONGLY-CONNECTED-COMPONENTS.



G

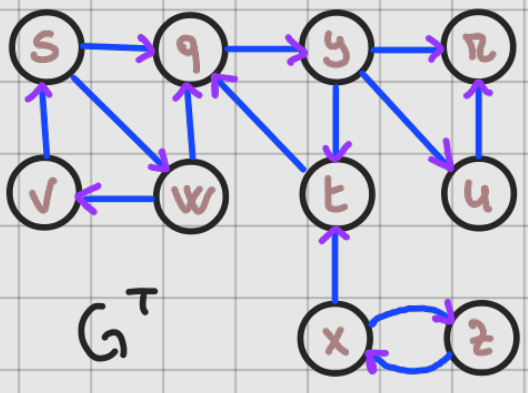
STRONGLY-CONNECTED-COMPONENTS(G)
1 call DFS(G) to compute finish times $u.f$ for each vertex u
2 create G^T
3 call DFS(G^T), but in the main loop of DFS, consider the vertices in order of decreasing $u.f$ (as computed in line 1)
4 output the vertices of each tree in the depth-first forest formed in line 3 as a separate strongly connected component

	v.f
q	16
r	20
s	7
t	15
u	19
v	6
w	5
x	12
y	14
z	11

1. Abbiamo già i finish times dall'es 6.

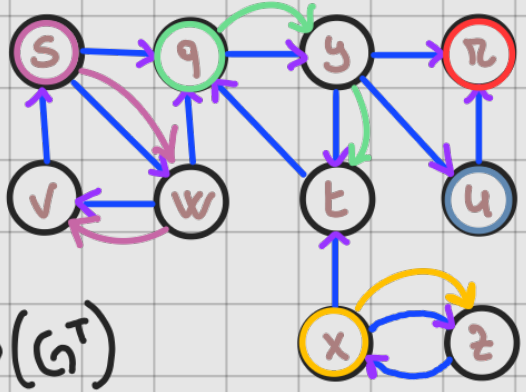
r u q t y x z s v w

2.



G^T

3.



DFS(G^T)

4.

